Remote Code Execution Classes

Educational Bug Bounty Series • Authorized Security Research Only

■ Remote Code Execution (RCE) — Vulnerability Classes & Defensive Review

Contributed by Cyber_Spy.404 | Educational Bug Bounty Series

Scope and ethics: *Review authorized code/configurations or use an isolated lab. Do not run exploit payloads, execute commands on third-party systems, or access internal services.*

■ Executive Summary

Remote Code Execution (RCE) occurs when an attacker causes a server to execute unintended code. The supplied two-page *rce.pdf* is a reading list covering dependency confusion, file uploads, SQL injection, LFI, SSRF, XXE, command injection, insecure deserialization, and server-side template injection.

■ Core Testing Methodology

1. Inventory uploads, dependency pipelines, database interfaces, file access, URL fetchers, XML parsers, deserializers, templates, and process calls.
2. Review package sources, namespace ownership, lockfiles, and CI/CD integrity.
3. Verify uploaded files cannot execute and are processed safely.
4. Check database operations use parameterized queries.
5. Constrain file paths and server-side network destinations.
6. Disable unsafe XML external entities.
7. Review command execution and avoid shell-string concatenation.
8. Review deserialization and template handling in code or an isolated lab.
9. Prefer configuration review, unit tests, and harmless canary markers.
10. Document the unsafe code path and retest remediation.

■ High-Value Bug Bounty Tips & Edge Cases

• Dependency confusion: use trusted registries, private namespaces, and pinned dependencies.

• Uploads: validate content and prevent execution from upload directories.

• SQL injection: parameterize queries and limit database privileges.

• LFI/SSRF: constrain paths and outbound requests; do not retrieve secrets or probe internal services.

• XXE: disable external entities and external DTD loading.

• Command injection: prefer language-native APIs.

• Deserialization: avoid untrusted native-object deserialization.

• SSTI: keep templates trusted and treat user input as data.

• The PDF is a list of links and categories, not a reproducible exploit guide.

■■ Educational Checklists & Safe Examples

```
Component: [Upload / Dependency / Query / URL Fetch / XML / Template]
Input source: [User-Controlled Field]
Suspected unsafe operation: [Code Path / Configuration]
Evidence: [Code Review / Sanitized Test]
Remediation: [Specific Change]
```

- [ ] Dependencies are pinned and trusted.
- [ ] Uploads cannot execute.
- [ ] Queries are parameterized.
- [ ] File and network access are constrained.
- [ ] XML external entities are disabled.
- [ ] Shell commands and unsafe deserialization are avoided.
- [ ] User input is not evaluated as template code.

■■ Mitigation & Remediation

Use trusted dependency registries and lockfiles, isolate uploads, parameterize queries, constrain file/network access, disable unsafe XML features, replace shell calls with native APIs, avoid unsafe deserialization, keep templates trusted, and apply least privilege and regression tests.

■ Source Notes

Based on the supplied two-page *rce.pdf*. Page 1 lists dependency confusion, file upload, SQL injection, LFI, SSRF, XXE, and command injection; page 2 lists insecure deserialization and SSTI. The source contains references rather than detailed procedures.

Educational use only. RCE testing requires explicit authorization and isolation.

Educational use only. Test only systems you own or are explicitly authorized to assess. Confirm redistribution rights for source-derived material.